Architectural Metabolism: Enforcing State Hygiene and Semantic Isolation in Repository-Resident Artificial Intelligence

1. Theoretical Framework of System Metabolism in AI Agent Workflows

The operation of repository-resident Artificial Intelligence (AI) agents necessitates a paradigm shift from traditional Continuous Integration/Continuous Deployment (CI/CD) pipelines toward a more biological model of "System Metabolism." In biological systems, metabolism represents the set of life-sustaining chemical reactions that convert inputs into energy for cellular processes and, crucially, eliminate metabolic byproducts to prevent toxicity. Similarly, an AI-augmented software development lifecycle requires rigorous mechanisms to ingest context (Dispatch Packets or DPs), execute complex logic, and systematically sanitize the operational environment upon completion. The accumulation of ephemeral artifacts, uncommitted changes, and "dirty" state variables constitutes a form of digital toxicity that degrades agent performance, induces hallucinations, and breaks the chain of custody required for auditability.

The execution of Dispatch Packet DP-OPS-0048, "System Metabolism Fixes," represents a critical maturation of this architecture. It transitions the system from manual, ad-hoc cleanup routines to automated, deterministic state resets. This analysis posits that a robust metabolic system must address three fundamental layers: the physical layer of ephemeral storage, the semantic layer of active contract management, and the cognitive layer of instruction isolation. The requirement to redirect temporary outputs from the operating system's volatile /tmp directory to a repository-controlled storage/_scratch location is not merely a path change; it is an assertion of sovereignty over the agent's memory model. It ensures that the repository remains the Single Source of Truth (SSOT), containing all artifacts necessary for forensic reconstruction while excluding "waste" that could contaminate future contexts.

Furthermore, the "Scrubbing" of the TASK.md work surface transforms the document from a static log into a dynamic state machine. By resetting fields to a neutral state after every cycle, we enforce a "Clean Room" protocol that prevents the leakage of previous intents into new sessions. This aligns with the "Anti-Drift Governance" mandates of the Policy of Truth (PoT), ensuring that every session begins with a verified, equilibrium state. Finally, protecting meta-instructions from inadvertent transcription addresses a specific failure mode of Large Language Models (LLMs): the inability to distinguish between the "content to be processed" and the "rules of processing." By establishing strict syntax barriers, we immunize the system

against the benign but disruptive form of prompt injection where an agent mistakenly treats its own operating instructions as payload data.

2. Ephemeral Storage Architecture: The Transition from System /tmp to Repository Scrubbing

The management of temporary files in automated workflows has traditionally relied on the operating system's /tmp directory hierarchy. However, in the context of high-fidelity AI agent operations, this reliance introduces significant vectors for instability, security vulnerability, and context leakage. The proposed redirection of all ephemeral outputs to storage/_scratch fundamentally alters the security posture and reliability of the ops/bin/prune worker and associated binaries.

2.1 The Volatility and Security Risks of System-Wide Temporary Storage

The standard Unix filesystem hierarchy standard (FHS) designates /tmp for temporary files that do not require preservation across reboots. This directory is typically world-writable, allowing any user or process on the system to create files.[1] For a repository-resident AI, utilizing /tmp breaks the containment model essential for deterministic behavior.

2.1.1 Context Leakage and "Orphaned" Artifacts

When an AI agent or a worker script like ops/bin/prune utilizes /tmp, the artifacts generated—such as intermediate diffs, context dumps, or temporary script payloads—exist outside the version-controlled boundary of the repository. If a process terminates unexpectedly or if the agent fails to execute its cleanup routine, these artifacts remain in /tmp as "orphans." These orphaned files often contain sensitive context from the repository, including code snippets, environment variables, or internal architectural diagrams.[2] Because they reside outside the repository tree, standard git-based hygiene tools (like git clean) cannot remove them, leading to a gradual accumulation of "dark data" on the host system. This data accumulation poses a risk of context contamination if a subsequent agent session accidentally ingests a stale file with a predictable name from a previous run.[3]

2.1.2 The Race Condition and Permission Vulnerabilities

The use of predictable naming conventions in /tmp (e.g., /tmp/stela-dump.txt) introduces Time-of-Check to Time-of-Use (TOCTOU) race conditions. A malicious actor or a concurrent process could pre-create a file with the same name, potentially tricking the agent into reading malicious input or overwriting critical data.[4] While the mktemp utility mitigates this by generating randomized filenames (e.g., tmp.XXXXXXXXXX), the underlying directory permissions of /tmp remain a concern in multi-user environments. By default, /tmp allows any user to write data, and while the "sticky bit" prevents deletion of others' files, it does not

inherently prevent visibility unless strict umask settings are enforced by the creating process.[2]

2.1.3 Persistence Mismatch and Debugging Complexity

System /tmp is frequently mounted as a tmpfs (RAM disk) or subject to aggressive automated cleanup policies (e.g., systemd-tmpfiles) that delete files after a short period or upon reboot.[5] This volatility is often desirable for true garbage, but the "System Metabolism" of an AI agent often requires a "persistent ephemeral" state. For instance, if an agent generates a massive "Context Snapshot" for debugging purposes, and the system reboots or the cleanup cron job runs, that forensic evidence is lost. Moving to storage/_scratch provides a localized, controlled persistence. Artifacts remain until the repository's own metabolic cycle (the prune script) decides to remove them, decoupling the agent's memory lifecycle from the host OS's uptime.[7]

2.2 Implementing Repository-Local Scratch with mktemp

The solution mandates a strict redirection of all temporary file creation to storage/_scratch. This directory is explicitly defined in the PoT.md Filing Doctrine as "Trash (local artifacts, never canon)" and is listed in .gitignore to prevent accidental commit.[8] Implementing this redirection requires manipulating the environment in which mktemp operates, ensuring compatibility across different operating systems (Linux GNU coreutils vs. macOS BSD).

2.2.1 The TMPDIR Environment Variable Strategy

The mktemp command, by default, utilizes the directory specified in the TMPDIR environment variable. If TMPDIR is unset, it falls back to /tmp.[1] While mktemp supports a -p flag (or --tmpdir in GNU versions) to specify a directory, the syntax and behavior vary significantly between BSD and GNU implementations.[9] The BSD mktemp (common on macOS) uses -t for a template path prefix, whereas GNU mktemp uses -t to imply utilizing TMPDIR.[10]

To achieve a POSIX-compliant, cross-platform solution that works reliably within the ops/bin/prune bash script, the most robust method is to export TMPDIR for the scope of the command or the script execution. This overrides the system default without requiring conditional logic to detect the OS type.

Bash

```
# Ensuring the scratch directory exists before use
SCRATCH_DIR="${REPO_ROOT}/storage/_scratch"
mkdir -p "$SCRATCH_DIR"

# Secure invocation of mktemp restricted to the repository scratch pad
```

```
# Using TMPDIR inline overrides the system default for this specific command
local tmp_file
tmp_file=$(TMPDIR="$SCRATCH_DIR" mktemp -t stela-prune.XXXXXX)
```

This approach ensures that mktemp creates the file inside storage/_scratch regardless of whether the system is an Ubuntu CI runner or a local macOS workstation.[12] The use of -t combined with the template ensures that a randomization string is appended, maintaining security against naming collisions within the scratchpad itself.[13]

2.2.2 Storage Hygiene and .gitignore Enforcement

The "Filing Doctrine" requires that storage/ contains only untracked artifacts. The existing .gitignore configuration already includes storage/_scratch/.[8] This effectively creates a "black hole" within the repository: files can be written there, read by agents, and deleted, but git status will remain blind to them. This is critical for maintaining the "Clean Porcelain" state required by the OPEN prompt.[8] If ops/bin/prune were to write to a tracked directory, it would immediately dirty the working tree, causing the "Freshness Gate" to fail in subsequent runs. By confining metabolic waste to storage/_scratch, we ensure that the "metabolism" does not interfere with the "anatomy" of the codebase.[14]

2.3 Forensic Advantages of Localized Scratch Space

Redirecting mktemp to storage/_scratch significantly enhances the forensic capabilities of the Operator. In the event of a script failure (e.g., ops/bin/prune crashes during a complex sed operation), the temporary files containing the intermediate state remain accessible in the repository structure.[7] An Operator can inspect storage/_scratch/stela-prune.a1b2c3 to see exactly what data was being processed at the moment of failure.

In contrast, failures involving /tmp are notoriously difficult to debug because the artifacts are often intermingled with system files or deleted by aggressive OS-level cleanup policies.[3] The ops/bin/prune script can be engineered to perform a "conditional scrub": it deletes the contents of storage/_scratch only upon successful completion of the cycle. If an error code is trapped, the scratch files are preserved, and the error message can point the Operator directly to the artifacts needed for diagnosis.[15]

3. Dynamic State Reset: The Logic of Scrubbing TASK.md

The TASK.md file acts as the "Active Contract" and "Living Surface" for the agentic workflow.[8] It stores the Session State (Branch, HEAD), the current Dispatch Packet metadata, and the chronological Work Log. While SoP.md serves as the immutable history ledger, TASK.md is the mutable short-term memory. A core failure in the current metabolism is the persistence of "stale" state in TASK.md after a task is completed. The next agent picks up the file and sees

the previous DP's ID and objectives, potentially leading to confusion or incorrect context loading.

The "Scrubbing" process is a semantic reset that transforms TASK.md from a "filled form" back into a "blank template" suitable for the next assignment. This requires a sophisticated text processing strategy that goes beyond simple file deletion.

3.1 The Semantic Structure of the Scrub

Scrubbing TASK.md involves identifying specific "Fields" within the Markdown structure and resetting them to default placeholder values (e.g., (none), (template), TBD). The challenge lies in the unstructured nature of Markdown; users or agents may introduce variations in whitespace, list formatting, or additional commentary that breaks rigid regex-based matching.[16]

The fields requiring reset are:

1. **Freshness Gate Pointers:** The Active Branch and Base HEAD fields must be reset to (none) or a generic placeholder. This forces the next session's ops/bin/open script to regenerate these values based on the *actual* git state at that time, rather than relying on stale data transcribed in the file.[8]
2. **Dispatch Packet Header:** The header ### DP-OPS-XXXX:... must be reverted to a generic ### DP-OPS-XXXX: (Template) or similar to indicate availability.
3. **Work Log Truncation:** The chronological entries under ## 5. Work Log must be deleted, leaving only the header. This prevents the Work Log from growing infinitely and confusing the model with history from unrelated tasks.

3.2 Text Processing Strategy: awk Record Processing vs. sed Stream Editing

While sed is the traditional tool for stream editing, it is often fragile when dealing with multi-line blocks or context-dependent replacements in structured text like Markdown.[17] sed operates line-by-line, making it difficult to define "regions" (e.g., "delete everything between header A and header B").

awk, specifically GNU awk (gawk), offers a more robust record-oriented processing model. It can maintain state variables (flags) to track which section of the document it is currently processing.[18] This allows for logic such as: "If inside the 'Work Log' section, discard lines; if inside 'Freshness Gate', replace values; otherwise, print lines as-is."

3.2.1 Implementing the Scrub Logic with awk

The ops/bin/prune script will utilize an awk filter to parse and rewrite TASK.md. This filter functions as a state machine:

Bash

```
awk '
  # State: Detect the "Current Dispatch Packet" section header
  /^## 3\. Current Dispatch Packet/ {
    print "## 3. Current Dispatch Packet (DP)"
    print "### DP-OPS-XXXX: (Template)"
    print ""
    skip_block = 1
    next
  }

  # State: Detect the start of the next section to exit "skip mode"
  /^### 3\.1 Freshness Gate/ {
    skip_block = 0
    print $0 # Print the header we just found
    next
  }

  # Action: Reset Freshness Gate values using regex for flexibility
  # Matches "- Active Branch: <anything>" and resets it
  /^([[:space:]]*)- Active Branch:/ {
    print "- Active Branch: (none)"
    next
  }
  /^([[:space:]]*)- Base HEAD:/ {
    print "- Base HEAD: (none)"
    next
  }

  # State: Detect Work Log header
  /^## 5\. Work Log/ {
    print "## 5. Work Log (Timestamped Continuity)"
    print "(No active thread)"
    exit # Stop processing entirely to truncate the rest of the file
  }

  # Default: Print line if not in a skip block
 !skip_block { print }
' "${REPO_ROOT}/TASK.md" > "${SCRATCH_DIR}/TASK_SCRUBBED.md" && mv
"${SCRATCH_DIR}/TASK_SCRUBBED.md" "${REPO_ROOT}/TASK.md"
```

This awk script is superior to sed because it is semantic. It reads the *sections* of the document

rather than just matching byte patterns.[19] It explicitly handles the truncation of the Work Log by using the exit command upon reaching the log header, ensuring essentially zero risk of leaving "ghost entries" from previous sessions.[20]

3.3 Integration with the Closeout Workflow

The placement of this scrub action within the ops/bin/prune execution flow is critical. It must occur *after* the "Log" phase of the Closeout Cycle defined in docs/MANUAL.md. The Closeout Cycle dictates the sequence: Verify -> Harvest -> Refresh -> Log -> Prune.[8]

If ops/bin/prune scrubs TASK.md before the metadata (DP ID, Objective) has been successfully logged to SoP.md, that provenance is lost forever. Therefore, ops/bin/prune acts as the "Finalizer" or "Destructor" of the DP object. The script effectively says: "The state has been saved to history (SoP.md); therefore, the working memory (TASK.md) can now be wiped." This ensures transactional integrity in the system's metabolism.

4. Cognitive Security: Protecting Meta-Instructions from Transcription

A pervasive failure mode in Large Language Model (LLM) workflows is "Instruction Leakage" or "Transcription Error." This occurs when the model reads a file containing both *instructions* (what to do) and *content* (what to process), and mistakenly interprets the instructions as content to be reproduced in the output.[21] The user's request highlights that the "Mandatory Closing Block" in TASK.md is being transcribed into the output instead of being treated as a logic constraint. This is a form of "Prompt Injection" where the system's own prompts act as the injection vector.[22]

4.1 The Psychology of the "Copy-Paste" Hazard

LLMs are trained to be helpful assistants. When they encounter text that looks like a form or a checklist (like TASK.md), their bias is to "complete" or "reproduce" it. The section "Mandatory Closing Block" often contains text like "Entry 4 must differ from Entry 1." To an LLM in "continuation mode," this looks like a sentence that should appear in the final report. The model fails to distinguish between the *meta-layer* (instructions to the model) and the *object-layer* (the text the model is generating).[23]

4.2 Mitigation Strategies: Markdown Syntax Guardrails

To prevent this, we must employ syntax patterns that signal "Metadata" to the model. We leverage the fact that LLMs trained on code repositories understand that certain syntactic elements (comments, frontmatter) are not part of the rendered output.

4.2.1 HTML Comment Encapsulation

The most robust defense is to wrap meta-instructions in HTML comments (``). In Markdown, HTML comments are valid syntax but are not rendered in the final document. LLMs generally understand that text inside these comments is intended for the "developer" (the model itself) and not the "user" (the final output).[24]

By wrapping the logic constraints in comments, we effectively "hide" them from the transcription attention head while keeping them visible to the logic attention head.

> **[Logic Gate]**
>
> *Condition:* Entry 4 must differ from Entry 1.
>
> *Action:* If identical, REWRITE Entry 4.

4.2.2 Explicit "Stop" Tokens and Blockquotes

In addition to comments, using blockquotes (>) combined with bold headers (e.g., > **[Logic Gate]**) creates a visual and structural distinction from the rest of the document text. This leverages the model's training on structured documentation where blockquotes often denote alerts or sidebars.[25]

We explicitly label these blocks as "Logic Constraints" rather than "Instructions," as "Instruction" can sometimes trigger the model's "following" behavior (copying the text), whereas "Constraint" implies a boundary condition.[26]

4.3 Prompt Engineering the Prune Script

The ops/bin/prune script serves as the enforcer of this protection. When it scrubs TASK.md, it shouldn't just leave blank lines; it should *inject* these guardrails. If a human operator accidentally deletes the comment tags while editing TASK.md, the next run of ops/bin/prune will regenerate the file from its internal template, restoring the protection. This makes the security posture "self-healing."

5. Hygiene and Compliance: Retaining and Hardening Previous Fixes

The DP-OPS-0048 mandate requires preserving the gains from previous architectural fixes: markdownlint compliance, dirty state exceptions, and receipt bundling. These are not isolated features but integral components of the metabolic cycle.

5.1 Dirty State Management: The TASK.md Exception

The PoT.md mandates a "Clean Porcelain" state, meaning git status --porcelain must be empty before a session begins.[8] However, TASK.md is the "Active Work Surface." It is *designed*

to be dirty; it changes the moment the operator logs an intent. Enforcing a clean git state on TASK.md creates a deadlock where the operator cannot update the task without committing, but cannot commit without finishing the task.

The ops/bin/open script captures the git state. The fix requires explicitly filtering TASK.md out of this check.

Bash

```
# Hardened logic for ops/bin/open and tools/verify.sh
# Check for uncommitted changes, IGNORING TASK.md
if]; then
  echo "CRITICAL: Working tree is dirty (excluding TASK.md)."
  echo "Please commit or stash changes before proceeding."
  exit 1
fi
```

This logic grants TASK.md a "diplomatic immunity" from the strict hygiene laws that govern the rest of the code, acknowledging its role as a meta-artifact rather than a source artifact.[14]

5.2 Mandatory Harvesting and Receipt Bundling

"Harvesting" is the metabolic function of extracting useful patterns (Skills, Agents) from the "waste" of a completed task. The ops/bin/prune script must verify that harvesting has occurred before destroying the artifacts. This effectively adds a "kidney" function: filtering out nutrients before excretion.

"Receipt Bundling" ensures that the dump command creates a single .tar.xz archive containing both the content (.txt) and the metadata (.manifest). This prevents "detached provenance," where a dump file exists without any record of what branch or commit it came from. The ops/bin/prune script must be updated to recognize these .tar.xz bundles as valid "Keepers" that should be moved to storage/archives/ rather than deleted.[8]

6. Technical Implementation Specifications

The implementation of DP-OPS-0048 centers on a major refactoring of ops/bin/prune and the TASK.md template.

6.1 Specification: ops/bin/prune v2.0 Logic

The new prune script acts as a "State Reset" utility.

Execution Flow:

1. **Environment Setup:** Define SCRATCH_DIR="${REPO_ROOT}/storage/_scratch" and create it if missing.
2. **Redirect Output:** Export TMPDIR="$SCRATCH_DIR" to force all sub-processes (like mktemp) to use the repository scratchpad.
3. **Active DP Identification:** Parse TASK.md using awk to extract the current DP-OPS-XXXX ID.
4. **Targeted Cleanup:**
   - Scan storage/handoff, storage/dumps, and storage/_scratch.
   - Identify files matching the current DP ID.
   - Move "Keepers" (defined as *-RESULTS.md and context-*.tar.xz) to storage/archives/.
   - Delete all other matching files.
5. **State Scrubbing:**
   - Pass TASK.md through the awk filter described in Section 3.2.1.
   - This resets "Active Branch" and "Base HEAD" to (none).
   - It resets the "Work Log" to a clean state.
   - It ensures the "Mandatory Closing Block" is wrapped in HTML comments.
6. **Final Report:** Output a summary of reclaimed space and the status of the TASK.md reset.

6.2 Specification: Hardened TASK.md Template

The TASK.md file must be updated to include the HTML comment guards natively.

Table 1: Hardened TASK.md Section Structure

Section	Current State	Hardened State	Protection Mechanism
Closeout	Plain text list	Comment-wrapped constraints	``
Gate	Plain text values	Values reset to (none)	awk script execution
Work Log	Persistent history	Truncated history	awk script execution

6.3 Addressing markdownlint Configuration

To permanently resolve the markdownlint warnings without disabling useful checks, we configure .markdownlint.json to ignore specific rules for the TASK.md file or globally if they conflict with the metabolic machinery.[28]

- **MD013 (Line Length):** Often triggered by the long base64 strings or URLs in the dump receipts. We disable this rule for storage/ and TASK.md or set a high character limit (e.g., 1000).[29]
- **MD025 (Multiple H1):** Can occur if the awk script accidentally appends a new header instead of replacing the old one. The fix is ensuring the awk logic uses explicit replacement (print...; next) rather than appending.[30]

7. Operational Ripple Effects and Conclusion

The implementation of these metabolic fixes establishes a "Clean Room" paradigm for the Stela repository. By enforcing the segregation of ephemeral storage to storage/_scratch, we eliminate the risk of cross-session contamination and ghost artifacts. The agents can operate with the confidence that the environment is pristine at the start of every task.[23]

The tension between "Auditability" (keeping records) and "Cleanliness" (deleting waste) is resolved by the "Bundle and Prune" strategy. We create permanent archives of the context and results before scrubbing the workspace. This satisfies the provenance requirements of the Policy of Truth without incurring the technical debt of a cluttered repository.

Ultimately, the self-healing nature of the scrubbed TASK.md reduces the cognitive load on the human operator. They no longer need to manually "clean up the mess" after an AI session; the ops/bin/prune command ensures that the repository metabolism functions autonomously, converting the chaotic byproducts of development into structured, archived history and a clean slate for the future.

Summary of Key Implementation Actions

1. **Modify ops/bin/prune:** Enforce TMPDIR=storage/_scratch for all operations.
2. **Implement awk Scrubber:** Replace manual resets with regex-based field clearing and log truncation.
3. **Harden TASK.md:** Wrap logic instructions in HTML comments to prevent transcription errors.
4. **Update PoT.md:** Formalize storage/_scratch as the mandatory temporary workspace for all agent operations.

This architecture ensures that Stela remains an "explainability-first" system, capable of sustaining high-tempo development cycles without succumbing to metabolic failure or state

corruption.

Works cited

1. Improving mktemp - Unix & Linux Stack Exchange, accessed February 11, 2026, https://unix.stackexchange.com/questions/415098/improving-mktemp
2. mktemp - Working with Temporary Files in Shell Scripts - Putorius, accessed February 11, 2026, https://www.putorius.net/mktemp-working-with-temporary-files.html
3. Quickly Create Temporary Files and Directories with mktemp - Bash Prompt, accessed February 11, 2026, https://bash-prompt.net/guides/bash-temp/
4. Creating temporary files in bash - Stack Overflow, accessed February 11, 2026, https://stackoverflow.com/questions/10982911/creating-temporary-files-in-bash
5. How To: Redirect Where ArcGIS pro Writes Temporary or Scratch Files during Processing, accessed February 11, 2026, https://support.esri.com/en-us/knowledge-base/how-to-redirect-where-arcgis-pro-writes-temporary-or-sc-000029492
6. Is it possible to change systemd private tmp directory? - Unix & Linux Stack Exchange, accessed February 11, 2026, https://unix.stackexchange.com/questions/435367/is-it-possible-to-change-systemd-private-tmp-directory
7. How do I create a temporary directory using mktemp in my current working directory? - Stack Overflow, accessed February 11, 2026, https://stackoverflow.com/questions/39519748/how-do-i-create-a-temporary-directory-using-mktemp-in-my-current-working-directo
8. dump-full-main-ca9b35f9.txt
9. How to create a temporary directory? - Stack Overflow, accessed February 11, 2026, https://stackoverflow.com/questions/4632028/how-to-create-a-temporary-directory
10. mktemp(1) - Linux manual page - man7.org, accessed February 11, 2026, https://man7.org/linux/man-pages/man1/mktemp.1.html
11. mktemp - Make a Temporary File or Directory - GNU.org, accessed February 11, 2026, https://www.gnu.org/software/autogen/mktemp.html
12. How to Use the mktemp Command to Create Temporary Files and Directories | Linode Docs, accessed February 11, 2026, https://www.linode.com/docs/guides/using-mktemp-command/
13. Intro To 'mktemp' Command In Linux - Robert Elder Software, accessed February 11, 2026, https://blog.robertelder.org/intro-to-mktemp-command/
14. Exclude files from git without committing changes to .gitignore | by Dave Lunny - Medium, accessed February 11, 2026, https://medium.com/@dave_lunny/exclude-files-from-git-without-committing-changes-to-gitignore-986fa712e78d
15. Local Scratch - Research Computing Documentation, accessed February 11, 2026,

https://docs.rc.uab.edu/data_management/cheaha_storage_gpfs/local_scratch/
16. Use awk or sed to remove everything between < > - Unix & Linux Stack Exchange, accessed February 11, 2026, https://unix.stackexchange.com/questions/652698/use-awk-or-sed-to-remove-everything-between
17. Reformat headers in Markdown files with sed fails - Stack Overflow, accessed February 11, 2026, https://stackoverflow.com/questions/43034248/reformat-headers-in-markdown-files-with-sed-fails
18. Markdown Rendering with Awk - Graham Marlow, accessed February 11, 2026, https://mgmarlow.com/words/2024-03-23-markdown-awk/
19. Remove empty sections in markdown with bash - Stack Overflow, accessed February 11, 2026, https://stackoverflow.com/questions/46729591/remove-empty-sections-in-markdown-with-bash
20. Awk commands find and replace string and print every thing after key word - Ask Ubuntu, accessed February 11, 2026, https://askubuntu.com/questions/1074853/awk-commands-find-and-replace-string-and-print-every-thing-after-key-word
21. LLM Prompt Injection Prevention - OWASP Cheat Sheet Series, accessed February 11, 2026, https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
22. The Invisible Threat: How Zero-Width Unicode Characters Can Silently Backdoor Your AI-Generated Code | Promptfoo, accessed February 11, 2026, https://www.promptfoo.dev/blog/invisible-unicode-threats/
23. Effective context engineering for AI agents - Anthropic, accessed February 11, 2026, https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
24. Prompt injection via markdown comments · community · Discussion #176693 - GitHub, accessed February 11, 2026, https://github.com/orgs/community/discussions/176693
25. AI Prompting Techniques | MindStudio University, accessed February 11, 2026, https://university.mindstudio.ai/1-core-building-principles/ai-prompting-techniques
26. Configure high-quality instructions for generative orchestration - Microsoft Copilot Studio, accessed February 11, 2026, https://learn.microsoft.com/en-us/microsoft-copilot-studio/guidance/generative-mode-guidance
27. Determine if Git working directory is clean from a script - Unix & Linux Stack Exchange, accessed February 11, 2026, https://unix.stackexchange.com/questions/155046/determine-if-git-working-directory-is-clean-from-a-script
28. markdownlint configuration in MegaLinter, accessed February 11, 2026,

https://megalinter.io/7/descriptors/markdown_markdownlint/
29. After installing markdownlint, why do the configuration rules not take effect? - Stack Overflow, accessed February 11, 2026, https://stackoverflow.com/questions/79712950/after-installing-markdownlint-why-do-the-configuration-rules-not-take-effect
30. Replace text between two marker with a file in bash - Stack Overflow, accessed February 11, 2026, https://stackoverflow.com/questions/77795269/replace-text-between-two-marker-with-a-file-in-bash